МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ 1

Основы Python и работа в VS Code и Zed

Вариант 13

по дисциплине «Программирование на языке Python»

Выполнил
Тебаев Мансур Дахирович
Группа ЭНЭ-б-о-25-2
Проверил _____

Ставрополь 2026

1 Тема и цель работы

Локальный проект: python-lab-01. Удалённый репозиторий не опубликован.

Тема работы — основы Python и работа в VS Code и Zed. В обоих разделах индивидуальных заданий выполнен вариант 13.

Цель работы — освоить создание и запуск программ Python, использование переменных, арифметических операций, консольного ввода и форматированного вывода; изучить выбор интерпретатора и исследование промежуточных значений при отладке.

2 Среда выполнения и структура проекта

Проверка программ выполнена в Windows 10 с интерпретатором Python 3.14.6. В папке проекта создано виртуальное окружение .venv. Учебные программы используют только встроенные средства Python и не требуют сторонних библиотек.

Проверено в VS Code 1.144.0 с Python Debugger 2026.6.0. Zed не запускался; его версия не фиксировалась.

Проект содержит примеры hello.py и zed_example.py, основные задания user.py, arithmetic.py, numbers.py, а также individual_01.py и individual_02.py. В README.md приведён порядок ввода и запуска. Файл verify.py проверяет программы, каталог results содержит результаты запусков. Виртуальное окружение исключено из Git файлом .gitignore.

Для подготовки окружения и проверки его версии используются команды:

```
python -m venv .venv
.\.venv\Scripts\python.exe --version
.\.venv\Scripts\python.exe -c "import sys; print(sys.executable)"
```

Полученная версия — Python 3.14.6. Путь интерпретатора оканчивается на python-lab-01\.venv\Scripts\python.exe, что подтверждает использование окружения проекта.

3 Ознакомительные примеры

3 1 Первый запуск hello.py

Переменной message присваивается строка. Функция print выводит её в стандартный поток вывода.

```
message = "Hello, Python!"
print(message)
```

Результат запуска:

```
Hello, Python!
```

3 2 Расчёт покупки zed_example.py

Количество тетрадей преобразуется в целое число, цена — в вещественное. Стоимость равна произведению количества на цену. Для одинакового формата запуска всех примеров значения читаются без приглашений к вводу.

```
quantity = int(input())
unit_price = float(input())


total_price = quantity * unit_price
print(f"Стоимость покупки: {total_price:.2f} руб.")
```

При вводе 3 и 25.5 получен результат:

```
Стоимость покупки: 76.50 руб.
```

Ручная проверка: 3 · 25,5 = 76,5. Формат .2f выводит два знака после десятичной точки и не изменяет само значение переменной.

4 Основные задания

4 1 Знакомство с пользователем user.py

Требуется ввести непустые строки имени и города, целый возраст от 0 до 120, затем вывести три строки с подписями. В проверках использованы вымышленные данные.

```
name = input()
age = int(input())
city = input()


print(f"Имя: {name}")
print(f"Возраст: {age}")
print(f"Город: {city}")
```

Если возраст только выводится, его можно оставить строкой. Преобразование int выбрано, чтобы возраст хранился как число; это необходимо для последующих арифметических действий или числового сравнения.

Результаты проверки приведены в таблице. Входные значения вводятся последовательно, каждое с новой строки.

Входные данные	Ожидаемый результат	Фактический результат
Анна; 18; Казань	Имя: Анна Возраст: 18 Город: Казань	Имя: Анна Возраст: 18 Город: Казань
Иван; 0; Омск	Имя: Иван Возраст: 0 Город: Омск	Имя: Иван Возраст: 0 Город: Омск
Елена; 120; Тула	Имя: Елена Возраст: 120 Город: Тула	Имя: Елена Возраст: 120 Город: Тула

Ожидаемые и фактические результаты совпали. Каждый запуск завершился с кодом 0; сообщений об ошибках нет.

4 2 Арифметический пример arithmetic.py

Программа предлагает вычислить выражение 4 · 100 − 54, самостоятельно получает правильный ответ и выводит его вместе с ответом пользователя. Условное сравнение ответов не требуется.

```
user_answer = input()
correct_answer = 4 * 100 - 54


print(f"Правильный ответ: {correct_answer}")
print(f"Ваш ответ: {user_answer}")
```

Умножение выполняется раньше вычитания: 4 · 100 − 54 = 400 − 54 = 346. Ответ пользователя оставлен строкой, поскольку он только выводится. Это позволяет отобразить и ответ «не знаю» без ошибки преобразования.

Результаты проверки приведены в таблице. Входные значения вводятся последовательно, каждое с новой строки.

Входные данные	Ожидаемый результат	Фактический результат
350	Правильный ответ: 346 Ваш ответ: 350	Правильный ответ: 346 Ваш ответ: 350
346	Правильный ответ: 346 Ваш ответ: 346	Правильный ответ: 346 Ваш ответ: 346
не знаю	Правильный ответ: 346 Ваш ответ: не знаю	Правильный ответ: 346 Ваш ответ: не знаю

Ожидаемые и фактические результаты совпали. Каждый запуск завершился с кодом 0; сообщений об ошибках нет.

4 3 Отношение сумм numbers.py

Вводятся четыре вещественных числа a, b, c, d, каждое с новой строки. Требуется вычислить (a + b) / (c + d). По условию c + d не равно нулю. Результат выводится с двумя знаками после точки.

```
a = float(input())
b = float(input())
c = float(input())
d = float(input())


numerator = a + b
denominator = c + d
result = numerator / denominator
print(f"Результат: {result:.2f}")
```

Суммы выделены в отдельные переменные numerator и denominator. Скобки определяют, что сначала вычисляются обе суммы, затем выполняется деление. В первом тесте 3 / 8 = 0,375, при выводе с .2f получается 0.38.

Результаты проверки приведены в таблице. Входные значения вводятся последовательно, каждое с новой строки.

Входные данные	Ожидаемый результат	Фактический результат
1; 2; 3; 5	Результат: 0.38	Результат: 0.38
-2.5; 1; 2; 1	Результат: -0.50	Результат: -0.50
-3; 3; 1; 1	Результат: 0.00	Результат: 0.00

Ожидаемые и фактические результаты совпали. Каждый запуск завершился с кодом 0; сообщений об ошибках нет.

5 Первое индивидуальное задание

Раздел «Индивидуальные задания 1», вариант 13. Входные данные: целые возрасты x, y от 0 до 120. Требуется найти средний возраст и модули отклонений каждого возраста от среднего.

Средний возраст m = (x + y) / 2. Отклонения равны |x − m| и |y − m|. Деление / сохраняет дробную часть. abs возвращает модуль разности; все три результата выводятся с двумя знаками после точки.

```
x = int(input())
y = int(input())

average_age = (x + y) / 2
deviation_x = abs(x - average_age)
deviation_y = abs(y - average_age)

print(f"Средний возраст: {average_age:.2f}")
print(f"Отклонение первого возраста: {deviation_x:.2f}")
print(f"Отклонение второго возраста: {deviation_y:.2f}")
```

Для x = 10 и y = 14 средний возраст равен 12, оба отклонения равны 2. Проверки охватывают границы диапазона, дробное среднее и равные возрасты.

Результаты проверки приведены в таблице. Входные значения вводятся последовательно, каждое с новой строки.

Входные данные	Ожидаемый результат	Фактический результат
10; 14	12.00; 2.00; 2.00	12.00; 2.00; 2.00
0; 120	60.00; 60.00; 60.00	60.00; 60.00; 60.00
17; 20	18.50; 1.50; 1.50	18.50; 1.50; 1.50
20; 20	20.00; 0.00; 0.00	20.00; 0.00; 0.00

В таблице значения приведены в порядке: средний возраст; первое отклонение; второе отклонение.

6 Второе индивидуальное задание

Раздел «Индивидуальные задания 2», вариант 13. Вводится положительное шестизначное число n. Требуется найти модуль разности суммы первых трёх цифр и суммы последних трёх цифр. Нулевой результат означает равенство сумм.

Использованы только числовые операции. Цифра извлекается делением на степень 10 и остатком от деления на 10. Преобразование числа в строку, ветвления и циклы не применяются. Допустимый диапазон: $100000 \leq n \leq 999999$.

```
n = int(input())

first_sum = n // 100000 + n // 10000 % 10 + n // 1000 % 10
last_sum = n // 100 % 10 + n // 10 % 10 + n % 10
difference = abs(first_sum - last_sum)

print(f"Модуль разности сумм: {difference}")
```

Для n = 100999 сумма первых трёх цифр равна 1 + 0 + 0 = 1, последних — 9 + 9 + 9 = 27. Модуль разности $|1 - 27|$ равен 26. Результат:

```
Модуль разности сумм: 26
```

Проверяются равенство сумм, разница 26 и границы диапазона. Во всех случаях результаты совпали; коды завершения равны 0.

Результаты проверки приведены в таблице. Входные значения вводятся последовательно, каждое с новой строки.

Входные данные	Ожидаемый результат	Фактический результат
123321	Модуль разности сумм: 0	Модуль разности сумм: 0
100999	Модуль разности сумм: 26	Модуль разности сумм: 26
100000	Модуль разности сумм: 1	Модуль разности сумм: 1
999999	Модуль разности сумм: 0	Модуль разности сумм: 0

7 Запуск и исследование промежуточных значений

Для запуска второго индивидуального задания из терминала проекта используется команда:

```
.\.venv\Scripts\python.exe individual_02.py
```

Контрольная трассировка Python при n = 100999 подтверждает следующие значения: перед строкой 5, вычисляющей difference, first_sum = 1 и last_sum = 27; перед строкой 7, выводящей результат, difference = 26. Это соответствует ручному выделению разрядов и вычислению модуля разности.

В VS Code 1.144.0 выполнена остановка на строке 5 individual_02.py при n = 100999: first_sum = 1, last_sum = 27. После команды шага выполнение остановилось перед строкой 7; difference = 26. Проверка проведена через Extension Host API с Python Debugger 2026.6.0; журнал — results/vscode_debug.json.

В Zed точку останова следует поставить на строке 5 individual_02.py. После команды debugger: start выбирается текущий файл Python и вводится 100999. Перед выполнением строки нужно проверить first_sum и last_sum, затем выполнить один шаг и проверить difference. Ожидаемые значения: 1, 27 и 26 соответственно. Точка останова приостанавливает выполнение перед строкой, поэтому difference до первого шага ещё не вычислена.

Описан порядок воспроизведения в Zed. Фактическую работу в редакторе и фиксацию его версии необходимо выполнить перед сдачей.

При трассировке numbers.py для входа 1; 2; 3; 5 перед делением получаются numerator = 3.0, denominator = 8.0; после деления result = 0.375. Отладка позволяет отличить ошибку вычисления от изменения представления числа при форматировании.

8 Ответы на вопросы для защиты

1 Редактор и интерпретатор

Редактор предназначен для ввода и изменения исходного кода, интерпретатор выполняет программу. В VS Code поддержку Python обеспечивают расширения Python и Python Debugger; в Zed используются языковые средства редактора, выбранный Python toolchain и адаптер отладки.

2 Версия и путь интерпретатора

Версия определяется командой python --version, путь — выражением sys.executable после import sys. На компьютере могут одновременно находиться системный Python, несколько установленных версий и интерпретаторы виртуальных окружений.

3 Назначение виртуального окружения

Окружение .venv изолирует установленные для проекта пакеты от других проектов и глобальной среды. В нём используется собственный путь запуска Python и собственный каталог пакетов; само окружение обычно не передаётся через Git.

4 Общее окружение двух редакторов

В VS Code выбирается Python: Select Interpreter, в Zed — toolchain: select. В обоих случаях указывается Python из одной папки .venv; совпадение путей sys.executable подтверждает использование одного интерпретатора.

5 REPL и запуск файла

В REPL команды вводятся и выполняются по одной, а значение выражения обычно показывается автоматически. При запуске .py-файла выполняется весь код; для явного вывода нужна print. Приглашение >>> означает ожидание следующей команды Python.

6 Присваивание и типы

Присваивание связывает имя с объектом, полученным справа от знака =. int представляет целые числа, float — числа с плавающей точкой, str — строки. Повторное присваивание может связать имя с объектом другого типа.

7 Результат input

input возвращает строку без завершающего перевода строки. int нужен для преобразования записи целого числа, float — для вещественного; без преобразования операция + над строками соединяет текст.

8 Деление и остаток

/ выполняет обычное деление, // — деление с округлением частного вниз, % — получение остатка. Поэтому −7 // 4 равно −2, а −7 % 4 равно 1; выполняется равенство $-7 = (-2) \cdot 4 + 1$.

9 Роль скобок в отношении сумм

В (a + b) / (c + d) скобки требуют сначала вычислить обе суммы. Без них выражение a + b / c + d сначала делит b на c и соответствует другой формуле.

10 Форматированные строки

В f-строке выражения в фигурных скобках заменяются их строковым представлением. Формат .2f выводит число в фиксированной форме с двумя знаками после точки; он влияет на вывод, а не присваивает переменной округлённое значение.

11 Параметры sep и end

sep задаёт разделитель между несколькими аргументами print, по умолчанию пробел. end задаёт строку после последнего аргумента, по умолчанию перевод строки. Например, print(1, 2, sep="; ", end="!") выводит 1; 2!.

12 Модуль math и единицы углов

import math даёт доступ к математическим функциям и константам, например math.sqrt и math.pi. Углы для sin, cos и tan задаются в радианах; для перевода градусов используется math.radians.

13 Точка останова и отладчик Zed

Точка останова временно останавливает выполнение перед выбранной строкой. В Zed для Python выбираются окружение и запуск через debugger: start. Затем в панели переменных проверяются значения, выполняется шаг или продолжается программа.

14 Выбор наборов для индивидуальных заданий

Для задания 1 использованы пары (10, 14), (0, 120), (17, 20), (20, 20): они проверяют пример, границы, дробное среднее и нулевые отклонения. Для задания 2 использованы 123321, 100999, 100000, 999999: проверяются равные суммы, разница 26 и границы диапазона.

15 Правила имён переменных

Имя может содержать буквы, цифры и подчёркивание, но не начинается с цифры и не совпадает с ключевым словом. Регистр различается, поэтому age и Age — разные имена. Имя input затенит встроенную функцию и затруднит дальнейший ввод.

16 Динамическая типизация

Тип принадлежит объекту, а не закрепляется за именем навсегда. Имя может последовательно ссылаться на объекты разных типов; type(value) возвращает тип текущего объекта.

17 Строковое и числовое сложение

Выражение "2" + "3" даёт строку "23" типа str. Выражение int("2") + int("3") даёт число 5 типа int, поскольку перед сложением обе строки преобразованы в целые числа.

18 Ввод восьми координат

Следует выполнить восемь отдельных присваиваний float(input()) для x1, y1, x2, y2, x3, y3, x4, y4. Значения 0, 0, 3, 0, 3, 2, 0, 2 вводятся по одному на строку; разделяющие запятые не вводятся.

19 Ошибки преобразования чисел

float ожидает десятичную точку, поэтому строка "2,5" не является допустимой записью вещественного числа Python. int принимает строковую запись целого числа, поэтому int("2.5") вызывает ValueError; при необходимости запись сначала преобразуется во float с отдельным решением о способе округления.

20 Точность вещественных вычислений

Десятичные дроби 0.1 и 0.2 не представляются точно в двоичном формате float. Поэтому их сумма может отображаться как 0.30000000000000004. Формат .2f показывает 0.30, не меняя исходный результат.

21 Приоритет арифметических операций

Сначала учитываются скобки, затем возведение в степень, унарные знаки, умножение и деление, далее сложение и вычитание. −2 ** 2 равно −4, (−2) ** 2 равно 4. Справа от ** допускается унарный минус: 2 ** −1 равно 0.5.

22 Цифры числа и перенос разрядов

Для положительного трёхзначного числа сотни равны n // 100, десятки — n // 10 % 10, единицы — n % 10. При сложении разрядов новая цифра равна сумме % 10, а перенос в следующий разряд — сумме // 10.

23 Переход через полночь и кольцевые позиции

Время удобно перевести в общее число минут или секунд. Для минут число полных суток равно total // 1440, время внутри суток — total % 1440. Новая позиция на кольце вычисляется как (position + shift) % size; положительный size обеспечивает результат от 0 до size − 1.

24 abs sqrt floor ceil

abs возвращает модуль; math.sqrt — неотрицательный квадратный корень; math.floor округляет вниз, math.ceil — вверх. При вещественных вычислениях аргумент math.sqrt должен быть неотрицательным, иначе возникает ValueError.

25 Чтение сообщений об ошибках

В traceback указываются файл и номер строки, последняя строка обычно содержит тип и причину ошибки. SyntaxError означает нарушение синтаксиса, ValueError — недопустимое значение для операции, ZeroDivisionError — деление на ноль. По этим сведениям находят и проверяют проблемный участок.

9 Вывод

Разработаны и проверены программы для консольного ввода, арифметических вычислений и форматированного вывода. В первом индивидуальном задании рассчитаны средний возраст и модули отклонений, во втором — разность сумм цифр шестизначного числа с использованием // и %. Все 19 проверок завершились успешно. Контроль промежуточных значений подтвердил соответствие программ используемым формулам.

10 Источники

Лабораторная работа № 1. Основы Python и работа в VS Code и Zed. Методические материалы курса. https://disk.yandex.ru/d/yT9RQ8LG9aQ_UA

Python 3.14. Учебник и стандартные типы. https://docs.python.org/3.14/tutorial/ и https://docs.python.org/3.14/library/stdtypes.html

Python в Visual Studio Code. https://code.visualstudio.com/docs/python/python-tutorial

Документация Zed по Python и отладке. https://zed.dev/docs/languages/python и https://zed.dev/docs/debugger